Exploring blind signatures under FHE by combining GBFV and HAWK (extended abstract)

Lorenzo Rovida
lorenzo.rovida@unimib.it

University of Milano-Bicocca, DISCo, Milan, Italy

Introduction

Blind signatures [Cha83] enable a third party to sign an encrypted message without having access to it. As an example use case, this enables the private signing of encrypted transactions in blockchain environments. One way to obtain such a primitive is, for instance, to evaluate the signature algorithm via homomorphic operations on the encrypted message. Recently, the HAWK [DPPW22] signature scheme has been introduced as a lightweight alternative to the already available FALCON [PFH+20]. The idea is to construct a similar hash-then-sign scheme á la [GPV08,HHGP+03], and HAWK does that by sampling points in (a rotation of) the $\mathbb{Z}^n$ lattice, hidden using a structured Lattice isomorphism problem (LIP) construction, called module-LIP. One of the main advantages of HAWK is its simplicity, as working over the $\mathbb{Z}^n$ lattice is much easier with respect to the NTRU lattice used in FALCON – think of, for example, discrete Gaussian samplers. Hopefully this simplicity can open the door to signing private messages using a fully homomorphic encryption (FHE) scheme.

Ideas and challenges

From a high-level perspective, given an FHE ciphertext containing some message $\boldsymbol{m} \in \{0,1\}^k$ for $k \in \mathbb{N}$, the following phases are required to sign it:

1. Add some uniformly random salt $\boldsymbol{r} \sim U(\{0,1\}^{\mathsf{saltlen}})$ to the message to get $(\boldsymbol{m} \,||\, \boldsymbol{r})$ and hash this concatenation to a fixed-size message to obtain $\boldsymbol{h} \leftarrow h(\boldsymbol{m} \,||\, \boldsymbol{r}) \in \{0,1\}^{2n}$, where n is the lattice dimension. In HAWK, they suggest performing this phase with $h \leftarrow$ SHAKE256 [Nat15] hash function, which allows control over the hash output size.
2. Given $\boldsymbol{h} \in \{0,1\}^{2n}$, fix a lattice point $\boldsymbol{t} \leftarrow B \cdot \boldsymbol{h}$, where B is a (secret) good basis for $\mathbb{Z}^n$.
3. Sample a coset of $\mathbb{Z}^{2n}$ from a discrete Gaussian sampler as $\boldsymbol{x} \leftarrow \mathcal{D}_{\mathbb{Z}^{2n}+\frac{1}{2}\boldsymbol{t},\sigma_{\text{sign}}}$.

The main obstacles are the hashing and the sampling from the $\mathcal{D}$ distribution.

Can [GV25] unlock practical hashing under FHE? There are not many examples of hash functions implemented under FHE, mainly due to their large computational cost. As a starting point, we will consider a proof-of-concept implementation focusing on the feasibility of such a circuit. It turns out that many permutation functions, such as Poseidon2 [GKS23], natively operate in the Goldilocks field (i.e., modulo $\Phi_6(2^{32}) = 2^{64} - 2^{32} + 1$) and are optimized for lightweight applications such as ZK and MPC use cases. A permutation function represents the main building block that can be used in different constructions (e.g., sponge construction) to obtain an actual hashing function. Luckily, the recent GBFV scheme by Geelen and Vercauteren [GV25] natively supports Goldilocks fields (among others) as the plaintext space, and additionally allows for parallel SIMD computations. This opens up many possible use cases since many cryptographic constructions are based on such field (see, e.g., [GBK+24]). Poseidon2, in particular, requires the following primitive operations:

- `matmul_external`/`matmul_internal`: optimized state-matrix multiplication where only additions and multiplications by constants are evaluated.
- `add_round_constants`: adding a round constant to the state.
- `s_box`: evaluating an S-box. In practice, this corresponds to $x \mapsto x^k$.

Notice that all procedures work in some field and are based on linear functions, making GBFV well-suited for such computations. The main bottleneck is represented by the S-boxes, where most of the multiplicative depth is consumed by computing powers of x. By a preliminary analysis, to evaluate Poseidon2 using the `POSEIDON2_GOLDILOCKS_8_PARAMS` set of parameters, the depth of the circuit, in terms of multiplications – ignoring optimizations – is of roughly 121 levels, of which 91 required by S-boxes only (assuming to evaluate x^7 with a tree approach which requires $\lceil \log(7) \rceil = 3$ levels). This parameters set allow to permute up to 6 values each of 63 bits, for a total of 378 bits. An open question is also about the most efficient way to transform such a permutation to an actual generic hash function. The most natural way would be to use a sponge construction, but this has the drawback of requiring many sequential operations – therefore deep circuits. Depending on the specific use case, this could still be feasible, e.g., in the signature of relatively short messages.

Signing the hash using HAWK. Assuming we are able to compute a fixed-size hash of a given message, we obtain $\boldsymbol{h} \leftarrow h(\boldsymbol{m} \,||\, \boldsymbol{s}) \in \mathbb{Z}^{2n}$, where $h(\cdot)$ is some hash function. We also assume a setting in which the key generation algorithm of HAWK has been run by the FHE scheme key owner, and the secret module-LIP polynomials (f, g, F, G) are encrypted under GBFV. The main obstacle is sampling from $\mathcal{D}$, which is the only nonlinear operation. Assuming access to (encrypted) uniformly distributed values, it could be possible to sample from $\mathcal{D}_{\mathbb{Z},\sigma}$ using a CDF function, although evaluating the latter under FHE is non-trivial. Depending on the specific use case, the user could also append an encrypted random sample along with the message to be signed, which could then be used as a "source of randomness". Another open question concerns data encoding,

since HAWK does not natively work under Goldilocks field and requires a binary input $\boldsymbol{h} \in \{0,1\}^{2n}$.

References

[Cha83] David Chaum. Blind signatures for untraceable payments. In *Advances in Cryptology – CRYPTO 1983*, pages 199–203, 1983.

[DPPW22] Léo Ducas, Eamonn W. Postlethwaite, Ludo N. Pulles, and Wessel van Woerden. Hawk: Module lip makes lattice signatures fast, compact andsimple. In *Advances in Cryptology – ASIACRYPT 2022*, pages 65–94, 2022.

[GBK+24] Mariana Gama, Emad Heydari Beni, Jiayi Kang, Jannik Spiessens, and Frederik Vercauteren. Blind zkSNARKs for private proof delegation and verifiable computation over encrypted data. Cryptology ePrint Archive, Paper 2024/1684, 2024.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version ofnbsp;thenbsp;poseidon hash function. In *Progress in Cryptology - AFRICACRYPT 2023: 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 1921, 2023, Proceedings*, page 177203, 2023.

[GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing – STOC 2008*, page 197206, 2008.

[GV25] Robin Geelen and Frederik Vercauteren. Fully homomorphic encryption forcyclotomic prime moduli. In *Advances in Cryptology – EUROCRYPT 2025*, pages 366–397, 2025.

[HHGP+03] Jeffrey Hoffstein, Nick Howgrave-Graham, Jill Pipher, Joseph H. Silverman, and William Whyte. NTRUSign: Digital Signatures Using the NTRU Lattice. In *Topics in Cryptology — CT-RSA 2003*, pages 122–140, 2003.

[Nat15] National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Technical Report FIPS PUB 202, U.S. Department of Commerce, 2015.

[PFH+20] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON. Tech. rep., National Institute of Standards and Technology, 2020. https://falcon-sign.info/falcon.pdf.